

Primera arquitectura cloud completa

Primera arquitectura cloud completa

Hoy no aprendes ningún servicio nuevo

Tienes ya todas las piezas sueltas: red con capas, instancias, almacenamiento y una base de datos sin credenciales en el código. Hoy las juntas en una sola arquitectura.

Piezas que ya conoces, juntas por primera vez

Front

S3, sitio estático

Aplicación

EC2, subred pública

Almacenamiento

EFS o S3, imágenes

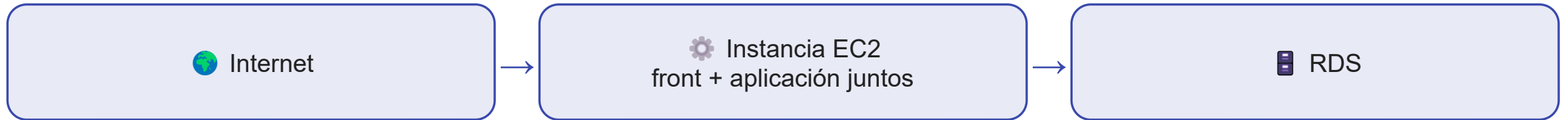
Base de datos

RDS, subred privada

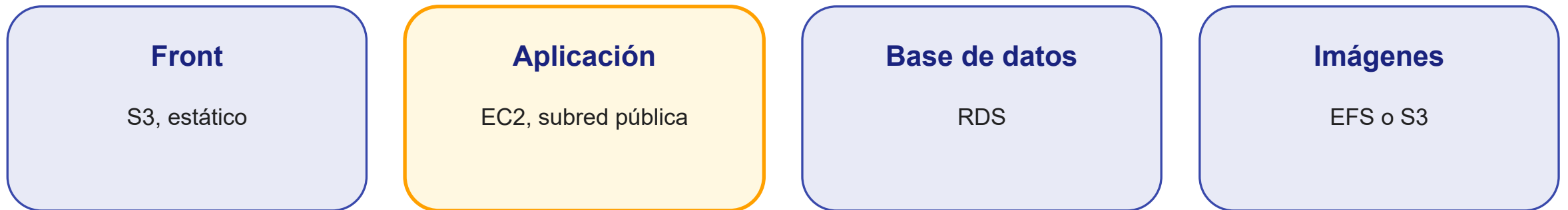
Es el cierre de todo lo construido desde la sesión 1, y el punto de partida de alta disponibilidad, monitorización, coste y automatización.

De dos piezas a tres capas independientes

Ahora: 2 piezas



Con lo de hoy: 3 piezas independientes



El front y la aplicación reciben tráfico de Internet por caminos distintos, y solo la aplicación tiene permiso para hablar con la base de datos y las imágenes — el mismo principio de mínimo privilegio de los grupos de seguridad del Tema 2, aplicado ahora a toda la arquitectura.

CORS: por qué el backend necesita permitirlo

El front vive en una URL de S3 y la aplicación en la IP de una EC2. Un origen se define por protocolo, dominio y puerto — con que uno solo cambie, ya son orígenes distintos.

	Protocolo	Dominio	Puerto
Front (S3)	http	escaparate-front-xxx.s3-website-...	80
Backend (EC2)	http	3.84.12.9	8080

Por defecto, un script cargado desde un origen no puede leer la respuesta de una petición hecha a otro origen. Esta restricción no protege al servidor: te protege a ti.

Por qué existe esta restricción: el banco y la pestaña maliciosa

Tienes el banco abierto en una pestaña, con sesión iniciada, y entras sin querer en una página maliciosa

1

Cualquier script puede pedir datos a cualquier URL, aunque no sea la suya.

2

El navegador guarda las cookies por dominio, no por página: si tienes sesión en el banco, cualquier petición a ese dominio lleva tu cookie puesta.

3

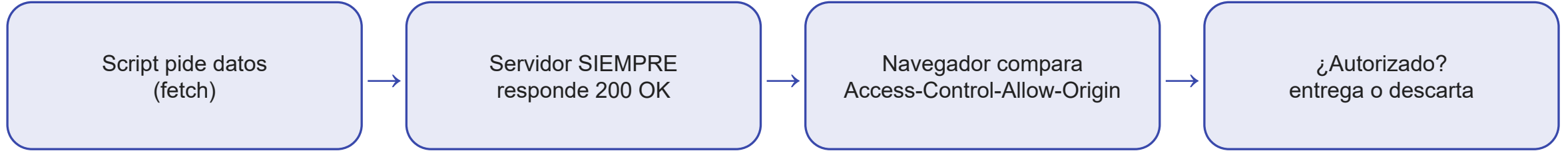
La petición del script malicioso llega al banco con tu cookie de sesión, y el banco responde como si fueras tú.

4

Aquí actúa la restricción: el navegador deja salir la petición, pero no deja que el script que la originó lea la respuesta.

CORS: cuándo se comprueba la cabecera

El servidor responde igual en los dos casos — nunca sabe si va a bloquear nada



Es el navegador quien, ya con la respuesta en la mano, decide si se la pasa al script o la descarta. Por eso, si abres la pestaña de red durante un fallo de CORS, ves la petición completada con código 200, y aun así el script se queda sin datos.

Cómo lo activas hoy: el código de Escaparate ya sabe leer qué origen autorizar mediante la variable `APP_CORS_ALLOWED_ORIGINS` — hoy le dices cuál es el origen real de tu bucket.

Separación de responsabilidades

Cada capa hace una cosa, y solo esa cosa

Capa	Responsabilidad	Lo que NO hace
Front (S3)	Servir HTML, CSS y JavaScript	No ejecuta lógica de negocio, no toca la base de datos
Aplicación (EC2)	Procesar peticiones, aplicar reglas de negocio	No sirve el front estático, no almacena datos de forma permanente
Base de datos (RDS)	Guardar y devolver datos de forma consistente	No sabe presentar nada, no toma decisiones de negocio

Si mañana necesitas escalar solo la aplicación porque hay más tráfico, no tocas ni el front ni la base de datos — esa independencia es la que hace posible el Tema 4.

Configuración externa

El endpoint de RDS y la contraseña de Secrets Manager no son dos mecanismos distintos: los dos son configuración externa. Ninguno vive escrito dentro del código.



Código de la aplicación
(fijo, no cambia)



Configuración externa
(variables de entorno)

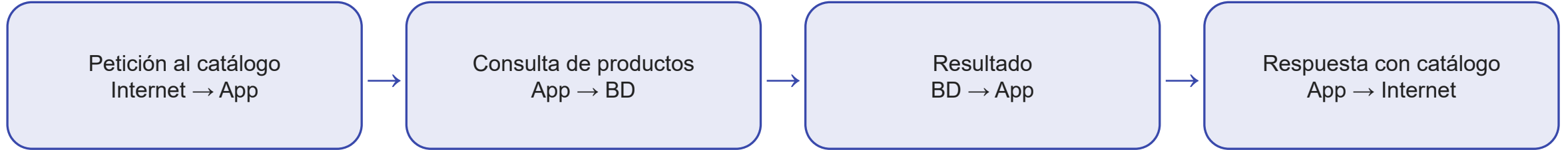
Entorno de producción

Entorno de pruebas

Si el endpoint viviera dentro del código, tendrías que modificar y redespargar toda la aplicación solo para apuntar a una base de datos de pruebas. Con configuración externa, el mismo artefacto sirve para los dos entornos.

Cadena de dependencias entre capas

Cada capa depende de que la de detrás esté disponible



Cuando algo no arranca a la primera

La solución casi nunca es cambiar el orden de creación de los recursos, sino hacer que la aplicación reintente la conexión unos segundos antes de darse por vencida, en vez de fallar a la primera.

Antes de rehacer nada: revisa el registro de arranque (/var/log/cloud-init-output.log) — casi siempre el mensaje ya te dice si el problema es la base de datos, la red o un permiso.

Qué se rompe cuando una pieza se mueve

Una arquitectura de capas separa responsabilidades, pero no las hace independientes del todo: si cambias el endpoint y no actualizas la configuración, la aplicación falla aunque la base de datos esté sana.

Qué se mueve	Qué se rompe si no se actualiza la referencia
Endpoint de la base de datos	La aplicación no puede conectar
Ruta del sistema de ficheros compartido	La aplicación no encuentra las imágenes de producto
Dirección del front	Los enlaces del catálogo hacia sus propios recursos dejan de resolver

Puntos únicos de fallo (SPOF)

Un punto único de fallo es cualquier pieza tal que, si falla ella sola, se cae el sistema entero. Una arquitectura resiliente sigue funcionando, aunque sea degradada, cuando algo falla.

Pieza	Por qué es un SPOF hoy
La instancia EC2 de la aplicación	Es una sola — si se para, no hay copia que responda en su lugar
La instancia RDS (sin Multi-AZ)	Sin copia en espera, un fallo de zona tumba la base de datos entera
La región donde despliegas todo	Un fallo a ese nivel se lleva por delante las tres capas a la vez

Tener SPOF hoy no es un error de diseño — es el punto de partida

Lo que hace madura una arquitectura no es no tener puntos únicos de fallo desde el primer día, sino saber nombrarlos y decidir cuáles resolver primero. Hoy los identificas; en el Tema 4 empiezas a eliminarlos.

Ideas clave

1

Tres capas —front (S3), aplicación (EC2), datos (RDS)— cada una con una única responsabilidad, sin saltarse ninguna: el mismo mínimo privilegio de los grupos de seguridad aplicado a toda la arquitectura.

2

Orígenes distintos (protocolo, dominio o puerto) hacen que el navegador bloquee por defecto que un script lea respuestas de otro — protege al usuario, no al servidor. CORS es la excepción explícita que concede el backend.

3

La separación de responsabilidades permite escalar o cambiar una capa sin tocar las demás.

4

La configuración externa hace que el mismo código sirva para varios entornos, sin credenciales ni endpoints fijos dentro del código.

5

Las capas dependen unas de otras en un orden concreto — si una referencia se queda apuntando al sitio viejo, la capa que depende de ella falla aunque la otra esté sana.

6

Un punto único de fallo tumba el sistema entero si falla solo — hoy hay al menos tres; identificarlos es el primer paso para resolverlos en las próximas sesiones.